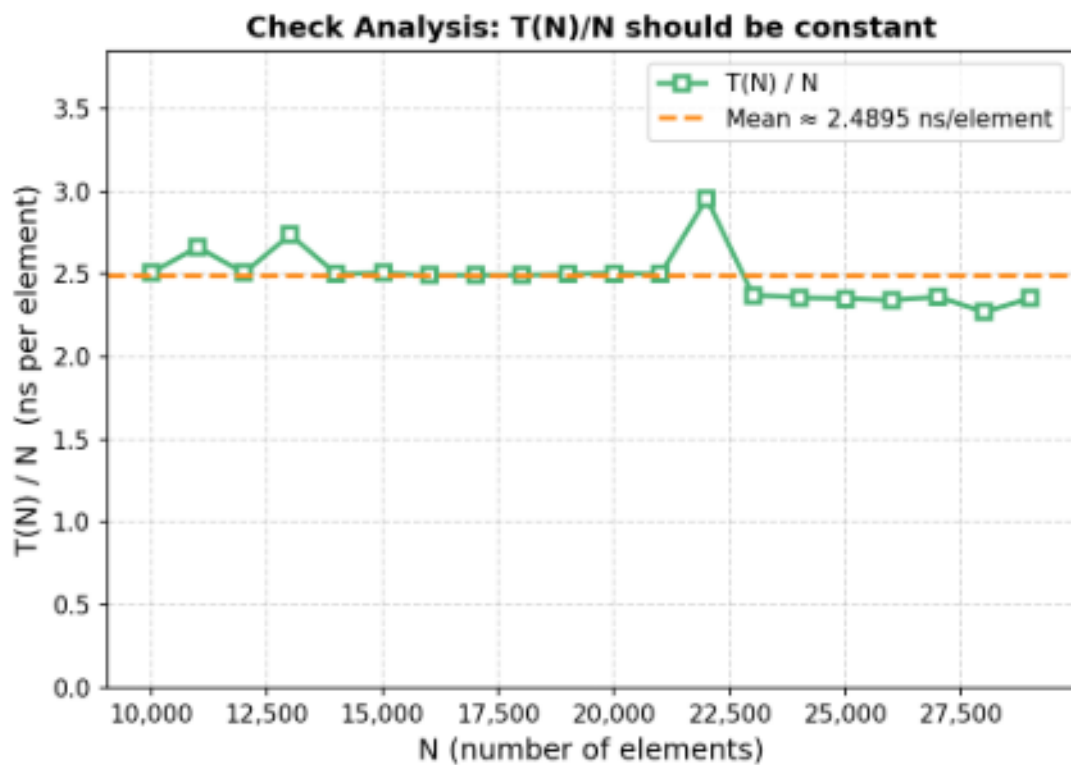
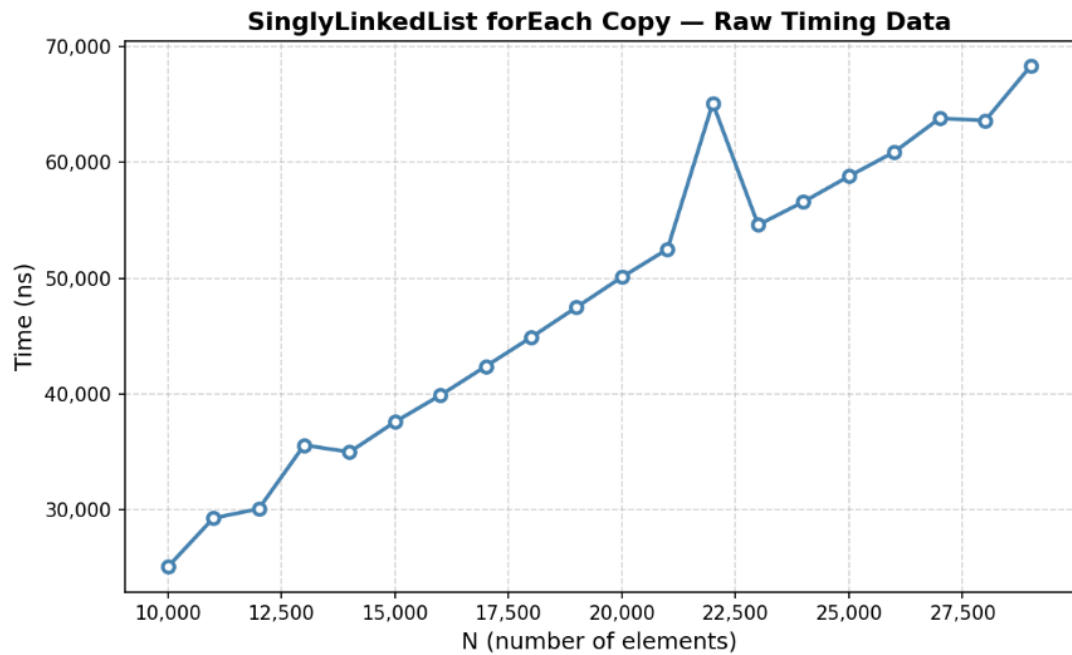


Assignment 6: LinkedListStack

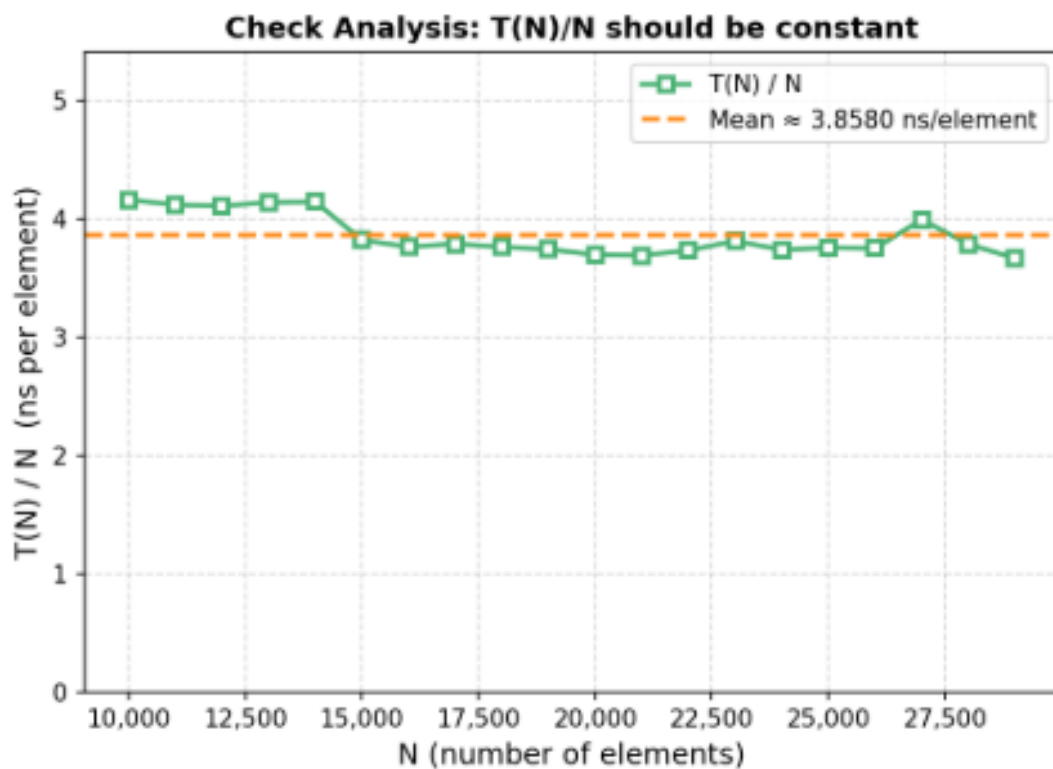
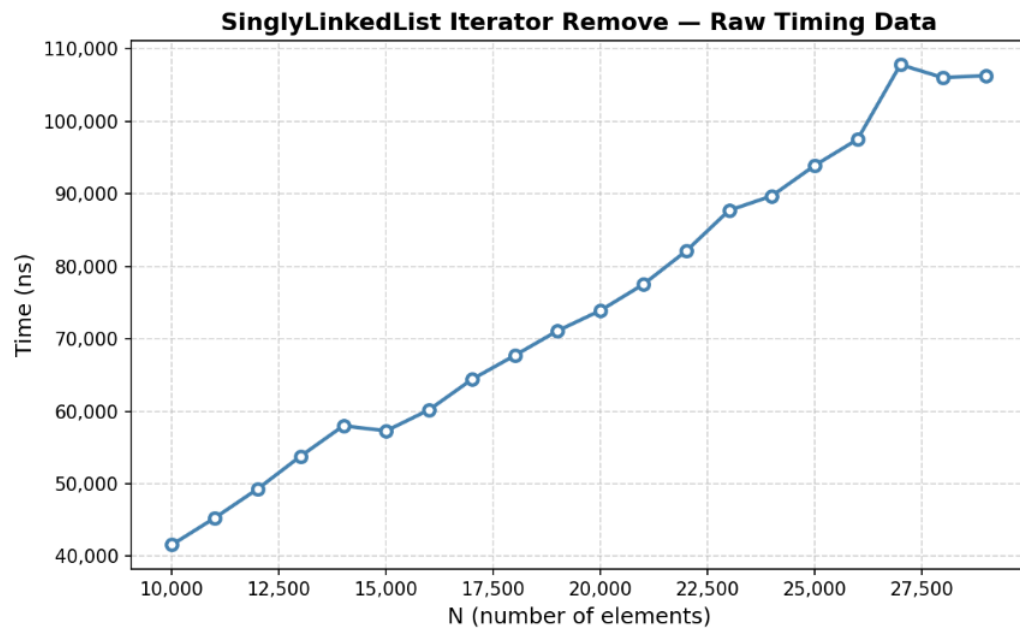
1. The growth rate of runtime is $O(N)$. This growth rate matches my expectations because when `copyLinkedListToArrayList` traversing the entire linked list using an enhanced for-loop, it actually calls an iterator `hasNext()` and `next()` method, and both methods have a time complexity of $O(1)$. Iterating through N elements requires calling the function N times, so the overall process is $O(N)$.



It can be observed that as the problem size increases, the ratio of runtime to the expected complexity becomes more and more like a constant, so whether the observed runtime matches the predicted runtime behavior is uncertain.

- The growth rate is $O(N)$. The growth rate meets my expectations because when `filterOutEvenNumbersFromLinkedList` during iteration, it actually calls `hasNext()`, `next()`, `remove()`. All three methods have a time complexity of

$O(1)$, and iterating through N elements requires calling N times, so the overall complexity is $O(N)$.

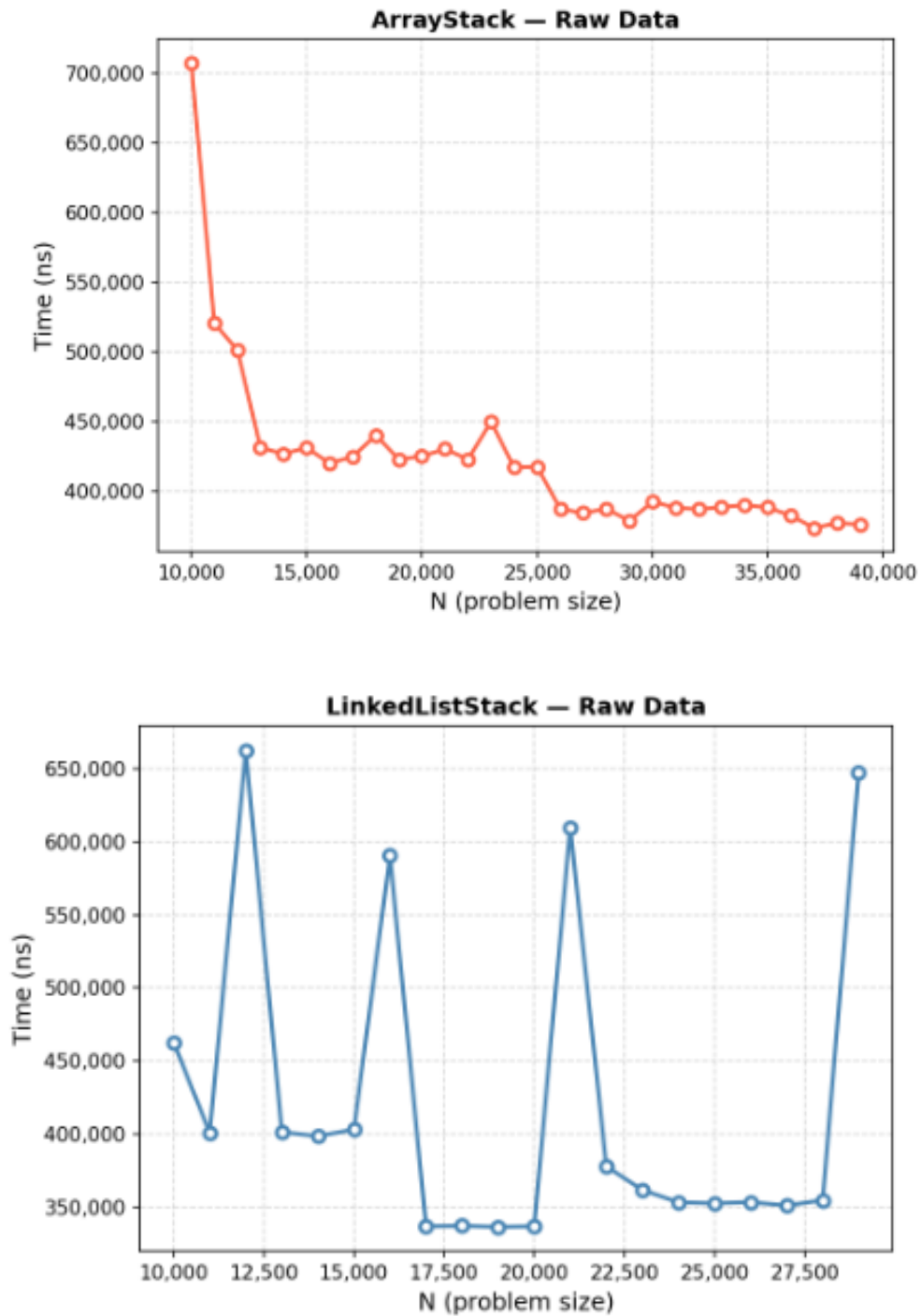


It can be observed that as the problem size increases, the ratio of runtime to the expected complexity becomes more and more like a constant, so whether the observed runtime matches the predicted runtime behavior is uncertain.

3. The iterator methods `hasNext`, `next`, and `remove`, these three methods cleverly minimize complexity to $O(1)$ by setting and moving pointers, thus achieving the fastest runtime when these methods are called multiple times in

timing experiments. Furthermore, these three methods do not create new list objects to consume memory. `hasNext()`: just a judgment `temp`. Whether the pointer is null has a time complexity of $O(1)$. `next()`: bundle `prevPrev`、`prev`、`temp` are moving the three pointers forward one step has a time complexity of $O(1)$. `remove()`: bundle `prevPrev.next` is directly pointing `temp` to skip the current node and the complexity is $O(1)$.

4. Both stacks have a growth rate of $O(N)$ for pushing N elements total. This matches my expectations: `LinkedListStack`'s `push` always calls `insertFirst`, which only involves pointer manipulation and runs in $O(1)$. `ArrayStack` adds a value to the end of the array each time, which is $O(1)$ in most cases. The only exception is when the backing array is full and resizing is triggered, which copies all elements and runs in $O(N)$. However, resizing happens rarely, so the amortized cost per push remains $O(1)$. Since both individual push operations are $O(1)$, pushing N elements in total results in $O(N)$ overall.



The check analysis confirms this: as N increases, the ratio $T(N)/N$ converges toward a constant value, which matches the predicted $O(N)$ behavior. The best-fit lines for both stacks have slopes close to the same value, further confirming that both grow linearly with N .

5. The ArrayStack stack class is more efficient at solving this problem. This is because LinkedListStack creates a new Node object every time a push operation is performed, which consumes extra time and memory. Therefore, the ArrayStack stack class is more efficient at resolving this issue.